

Práctica 0

Grupo Q

Marc Martínez Arias, Pedro Barros Bobadilla

[URL del repositorio](#)

Tenemos que diseñar una función que encuentre un hash dorado. Este hash dorado va a ser aquel hash que los últimos 6 dígitos sean 0. Además tenemos que compararlo y saber la distancia Manhattan respecto a las coordenadas iniciales dadas, la función aleatorio se va a encargar de esa tarea.

```
In [1]: #Importamos las librerías necesarias
import random as rd
import time
import threading
import multiprocessing
from multiprocessing import Value, Array

# Funciones auxiliares

#Definimos una función que encuentre valores aleatorios
def aleatorio(x:int,y:int) -> tuple:
    x,y = rd.choice([(1,0),(-1,0),(0,1),(0,-1)])
    return x,y

# Definimos otra función que comprueba si un hash es dorado
def hash_dorado(hash:int) -> bool:
    return hash % 1000000 == 0
```

Lupas

Ahora vamos a programar la función lupa que se va a centrar principalmente en encontrar la coordenada que tenga el hash dorado más cercano a la coordenada que se da a la función. Además, también devolverá la coordenada y el número de iteraciones que ha hecho para encontrar el hash dorado.

```
In [2]: def lupa(coordenadas:tuple[int,int]) -> tuple[int,int]:
    # Creamos una variable vacía para guardar las iteraciones
    x0, y0 = coordenadas
    x, y = x0, y0
    iteraciones = 0
    # Control de errores
    if not isinstance(coordenadas,tuple):
        raise ValueError("El valor ha de ser una tupla")
    if len(coordenadas) != 2:
        raise ValueError("La tupla ha de tener 2 únicos valores")
    # Definimos el algoritmo principal
    while True:
        hash_actual = hash((x, y))
        iteraciones += 1
        if hash_dorado(hash_actual) == True:
```

```

        manhattan = abs(x - x0) + abs(y - y0)
        if manhattan <= iteraciones:
            return x, y

    # Sumamos una coordenada manhattan y repetimos el proceso
    dx,dy = aleatorio(x,y)
    x += dx
    y += dy

```

Ahora vamos a ejecutar 8 lupas a la vez de dos formas diferentes y vamos a comprobar cual es la más rápida. Por una parte de forma secuencial empezando por coordenadas separadas y por otra parte de forma concurrente utilizando hilos de ejecución

In [3]: # SECUENCIAL

```

def lupa_secuencial(lupas:int):
    # Generamos unas coordenadas para comprobar que forma es más rápida
    coordenadas = []
    for i in range(1,lupas+1):
        coordenadas.append((i*1000,i*1000))
    # Iniciamos el tiempo para calcular la velocidad
    init = time.time()
    # Generamos el algoritmo
    resultados = []
    for coord in coordenadas:
        resultado = lupa(coord)
        resultados.append(resultado)
    # Calculamos el tiempo y devolvemos los resultados
    end = time.time()
    total_time = round(end-init,5)
    return total_time, resultados

```

CONCURRENCIA

```

def lupa_concurrente(lupas:int):
    # Generamos unas coordenadas para comprobar que forma es más rápida
    coordenadas = []
    for i in range(1,lupas+1):
        coordenadas.append((i*1000,i*1000))
    resultados = [None] * lupas
    #Definimos el worker para los threads
    def worker(idx, coord):
        resultados[idx] = lupa(coord)
    # Iniciamos el tiempo para el algoritmo
    init=time.time()
    # Desarrollamos el algoritmo
    hilos = []
    for i, coord in enumerate(coordenadas):
        hilo = threading.Thread(target=worker, args=(i, coord))
        hilos.append(hilo)
        hilo.start()
    # Desarrollamos la parte concurrente
    for hilo in hilos:
        hilo.join()
    # Calculamos el tiempo y devolvemos el resultado
    end = time.time()
    total_time = round(end - init,5)

```

```
return total_time, resultados
```

Vamos a comprobarlo mediante un ejemplo.

```
In [4]: print("Secuencial:")
t_sec, res_sec = lupa_secuencial(4)
print(f"Tiempo: {t_sec}s")
print(f"Resultados: {res_sec}\n")

print("Concurrente:")
t_conc, res_conc = lupa_concurrente(4)
print(f"Tiempo: {t_conc}s")
print(f"Resultados: {res_conc}")
```

```
Secuencial:
Tiempo: 8.73922s
Resultados: [(402, -584), (824, 1377), (2672, 3105), (4211, 3995)]
```

```
Concurrente:
Tiempo: 37.4846s
Resultados: [(570, 817), (3161, -4614), (2418, 2545), (4211, 3995)]
```

Como vemos de forma secuencial tarda menos. Esto es por culpa del GIL. Dado que todos los hilos comparten el mismo interprete de python el GIL permite que solo se ejecute un hilo a la vez, así que los demás están bloqueados hasta que se termine el proceso del hilo. A diferencia del secuencial que se ejecutan todos a la vez. Sin el GIL el concurrente si sería mucho más veloz.

Concursos

Ahora vamos a pasar a la parte del concurso, que se centra principalmente en desarrollar un programa concurrente que "lance" 8 lupas a la vez y termine en cuanto una lupa encuentre un hash dorado.

```
In [5]: def worker_lupas(idx, coord, q):
        try:
            resultado = lupa(coord)
            q.put((idx, resultado))
        except Exception as e:
            # si un proceso falla, mandamos el error para que el padre no se
            q.put((idx, e))

def lupas_procesos(n_lupas: int = 8, separacion: int = 1000):
    coordenadas = [(i*separacion, i*separacion) for i in range(1, n_lupas)]
    q = multiprocessing.Queue()

    init = time.time()

    procesos = []
    for i, coord in enumerate(coordenadas):
        p = multiprocessing.Process(target=worker_lupas, args=(i, coord,
                                                                q))
        procesos.append(p)
        p.start()

    resultados = [None] * n_lupas
    for _ in range(n_lupas):
```

```

        idx, dato = q.get()    # ahora SIEMPRE llega algo (resultado o exc
        resultados[idx] = dato

    for p in procesos:
        p.join()

    end = time.time()
    return round(end - init, 5), resultados

if __name__ == "__main__":
    multiprocessing.freeze_support()

    print("Procesos:")
    t_proc, res_proc = lupas_procesos(8)
    print(f"Tiempo: {t_proc}s")
    print(f"Resultados: {res_proc}")

```

Procesos:

Tiempo: 34.60813s

Resultados: [(1748, 2241), (5397, -6501), (2204, 5526), (-3633, 4294), (5110, 3943), (5349, 5360), (8448, 6094), (7056, 7651)]

```

In [6]: def worker_concurso(idx, coord, q, evento):
        try:
            resultado = lupa(coord)
            if not evento.is_set():
                evento.set()
                q.put((idx, coord, resultado))
        except Exception as e:
            # opcional: si falla, avisamos (no debería)
            q.put((idx, coord, e))

    def concurso_procesos(n_lupas: int = 8, separacion: int = 1000):
        coordenadas = [(i*separacion, i*separacion) for i in range(1, n_lupas)]
        q = multiprocessing.Queue()
        evento = multiprocessing.Event()

        init = time.time()

        procesos = []
        for i, coord in enumerate(coordenadas):
            p = multiprocessing.Process(target=worker_concurso, args=(i, coord, q, evento))
            procesos.append(p)
            p.start()

        idx, coord0, resultado = q.get()    # primer ganador (o error)

        for p in procesos:
            if p.is_alive():
                p.terminate()
        for p in procesos:
            p.join()

        end = time.time()
        return round(end - init, 5), {"idx": idx, "coord_inicio": coord0, "co

if __name__ == "__main__":
    multiprocessing.freeze_support()

```

```
print("Concurso con procesos:")  
t_conc, ganador = concurso_procesos(8)  
print(f"Tiempo: {t_conc}s")  
print(f"Ganador: {ganador}")
```

Concurso con procesos:

Tiempo: 0.60383s

Ganador: {'idx': 3, 'coord_inicio': (4000, 4000), 'coord_final': (3755, 4192)}